Databases Project Report

Group members (names and student numbers):

- Vo Dinh Nhan - 103773920
- Nguyen Vo Trong Nguyen - 103763778
- Nguyen Viet Dung - 103778682
- Tran Hoang Minh - 103561860
- Nguyen Quang Huy - 103770428

Subject Analysis - Room Booking Database

1. Candidate Subjects

Entities and Key Attributes

Building

Attribute	Type	Notes
id	PK	Primary key
name	NOT NULL	Building name
address	NOT NULL	Physical address

Room

Attribute	Type	Notes
id	PK	Primary key
building_id	FK	-> Building
name_or_number	NOT NULL	Room identifier within the building
capacity	NOT NULL	Maximum number of people
type	NOT NULL	meeting room / lecture room / studio / workshop room
requires_approval	NOT NULL	Boolean - whether this room requires special access approval

EquipmentType

Attribute	Type	Notes
id	PK	Primary key
name	NOT NULL	e.g. projector, whiteboard, microphone, video camera

RoomEquipment (linking table)

Attribute	Type	Notes
room_id	FK	-> Room
equipment_id	FK	-> EquipmentType
quantity	NOT NULL	Number of equipments

Organization

Attribute	Type	Notes
id	PK	Primary key
name	NOT NULL	Student organization name

Person

Attribute	Type	Notes
id	PK	Primary key
name	NOT NULL	Full name
email	NOT NULL	Contact email - minimum personal data only

BookingSeries (the planning intent for recurring bookings)

Attribute	Type	Notes
id	PK	Primary key
room_id	FK	-> Room
organization_id	FK, nullable	-> Organization; NULL if series is created by an individual with no organizational affiliation
created_by	FK	-> Person
recurrence_rule	NOT NULL	e.g. "weekly", "biweekly"
series_start_date	NOT NULL	First occurrence date
series_end_date	NOT NULL	Last occurrence date
day_of_week	NOT NULL	e.g. "Wednesday"
start_time	NOT NULL	Time of day the slot starts
end_time	NOT NULL	Time of day the slot ends

Why this table exists: The case explicitly requires that a recurring booking be treatable *"both as a single planning intent and as a series of individual occurrences."* `BookingSeries` stores that intent. Without it, there is no single fact in the database representing "the organization has booked Wednesday evenings." The pattern fields (`recurrence_rule`, `day_of_week`, `start_time`, `end_time`) belong here because they describe the original intent, not any individual occurrence.

Booking (one time slot - standalone or one occurrence of a series)

Attribute	Type	Notes
id	PK	Primary key
room_id	FK	-> Room

Attribute	Type	Notes
`organization_id`	FK, nullable	-> Organization; NULL if booking is made by an individual with no organizational affiliation
`created_by`	FK	-> Person
`start_datetime`	NOT NULL	Start of the booking slot
`end_datetime`	NOT NULL	End of the booking slot
`status`	NOT NULL	One of: ‘pending’ ‘confirmed’, ‘cancelled’, ‘rejected’
`series_id`	FK, nullable	-> BookingSeries; NULL if standalone booking
`created_at`	NOT NULL	Timestamp when the booking was created - needed for reporting and auditing
`cancelled_at`	nullable	Timestamp of cancellation; NULL if not cancelled
`cancellation_reason`	nullable	Short free text; NULL if not cancelled
`requires_approval`	NOT NULL	Snapshot of `Room.requires_approval` at booking creation time
`approval_granted`	nullable	NULL if approval not required or not yet decided; TRUE if approved; FALSE if rejected

Why `requires_approval` appears on both Room and Booking: `Room.requires_approval` reflects the room's current policy. `Booking.requires_approval` is a snapshot taken at booking creation time. If the room's policy changes later, historical bookings are not affected. This is a justified denormalization, noted in the design-quality review.

Reading approval state:

requires_approval	approval_granted	Valid Statuses
FALSE	NULL	Any (default pending, but also confirmed, cancelled, rejected)
TRUE	NULL	Pending, cancelled
TRUE	TRUE	Confirmed, Cancelled
TRUE	FALSE	Rejected

2. Candidate Relationships

From	Relationship	To	Cardinality	Notes
Building	contains	Room	1 to many	A room belongs to exactly one building

From	Relationship	To	Cardinality	Notes
Room	has	EquipmentType	many to many	Via `RoomEquipment` linking table
Person	creates	Booking	1 to many	Minimal personal data stored
Person	creates	BookingSeries	1 to many	Same person model used for series creation
Organization	associated with	Booking	0 or 1 to many	Organization linked via FK; nullable - a booking may have no organization if made by an individual
Organization	associated with	BookingSeries	0 or 1 to many	Organization linked via FK; nullable - a series may have no organization if created by an individual
Room	is booked by	Booking	1 to many	A booking covers exactly one room
Room	is target of	BookingSeries	1 to many	A series targets one room
BookingSeries	consists of	Booking	1 to many	A standalone booking has `series_id = NULL`

3. Key Assumptions

1. **Equipment is modeled as types, not physical assets.** A room either has a projector or does not. Serial numbers and individual item tracking are out of scope.
2. **A standalone booking and a recurring occurrence share the same** Booking table. A standalone booking simply has series_id = NULL.
3. `requires_approval` **is snapshotted onto** `Booking` **at creation time.** This ensures historical bookings are not affected if a room's approval policy changes later.
4. `approval_granted` **on** `Booking` **covers both the "required" and "outcome" states** via a nullable boolean. No separate approval table, no approver identity, no workflow stages - the case only asks whether approval was required and whether it was granted.
5. **Organization is a separate entity**, not just a name string on the booking. This makes reporting on organizations clean and consistent.
6. `organization_id` **is nullable on both** `Booking` **and** `BookingSeries`**.** A booking or a recurring series may be made by an individual with no organizational affiliation. This applies equally to standalone and recurring bookings - a person should be able to book a room for themselves in both cases.

7. **Person stores name and email only.** This is the minimum needed to manage bookings responsibly, as stated in the case.
8. **Double-booking is prevented** - a room cannot have two confirmed bookings with overlapping time slots. This is enforced at the schema or application level.
9. **Recurrence is stored as a descriptive rule string and date range** (e.g. "weekly", start date, end date, day of week). The database does not parse or enforce the rule - it is descriptive only.
10. `created_at` **is recorded on every Booking** to support reporting (e.g. bookings made per month) and auditing.

4. Ambiguities and Chosen Interpretations

Ambiguity	Chosen Interpretation
Is equipment a specific physical item or a type?	Equipment type per room. No serial numbers or individual asset tracking.
Does a recurring booking need its own table?	Yes - `BookingSeries` stores the planning intent; individual `Booking` rows are the occurrences. Required explicitly by the case description.
What personal data is stored about the person creating a booking?	Name and contact email only.
Can the same room be double-booked?	No. Enforced as a constraint on confirmed bookings.
What does "approval" mean in practice - workflow, approver identity?	We store only `requires_approval` (snapshot boolean) and `approval_granted` (nullable boolean). No approver identity or multi-step workflow.
Can a cancellation reason contain personal information?	Possibly. We store it as a short optional free-text field and flag it as a privacy risk in the design-quality review.
Is recurrence pattern always weekly, or flexible?	We store a `recurrence_rule` string (e.g. `"weekly"`, `"biweekly"`) and a date range. The design does not enforce or parse the rule.
Is organization a separate entity or just a name string?	Separate entity with its own table, to support clean reporting by organization.
Should `requires_approval` live only on Room or also on Booking?	Both - `Room.requires_approval` is the current policy; `Booking.requires_approval` is a snapshot at creation time. Justified denormalization.
Can a booking be made without an organization?	Yes for both standalone bookings and recurring series - `organization_id` is nullable on both `Booking` and `BookingSeries`. A person may book a room for themselves without any organizational affiliation.

5. Likely Application Queries

These are the queries the schema must be able to answer. All must be traceable through the conceptual model.

ID	Query	Source
Q1	How many bookings has each room had?	Case description
Q2	Which organizations use the service most?	Case description
Q3	Which rooms are most heavily used (by total hours booked)?	Case description
Q4	Which equipment types appear most often in booked rooms?	Case description
Q5	How many bookings were cancelled?	Case description
Q6	Which bookings required approval, and was it granted?	Case description
Q7	List all occurrences of a given booking series	Derived from recurring booking requirement
Q8	Which bookings were made by a given organization within a date range?	Derived from reporting requirement

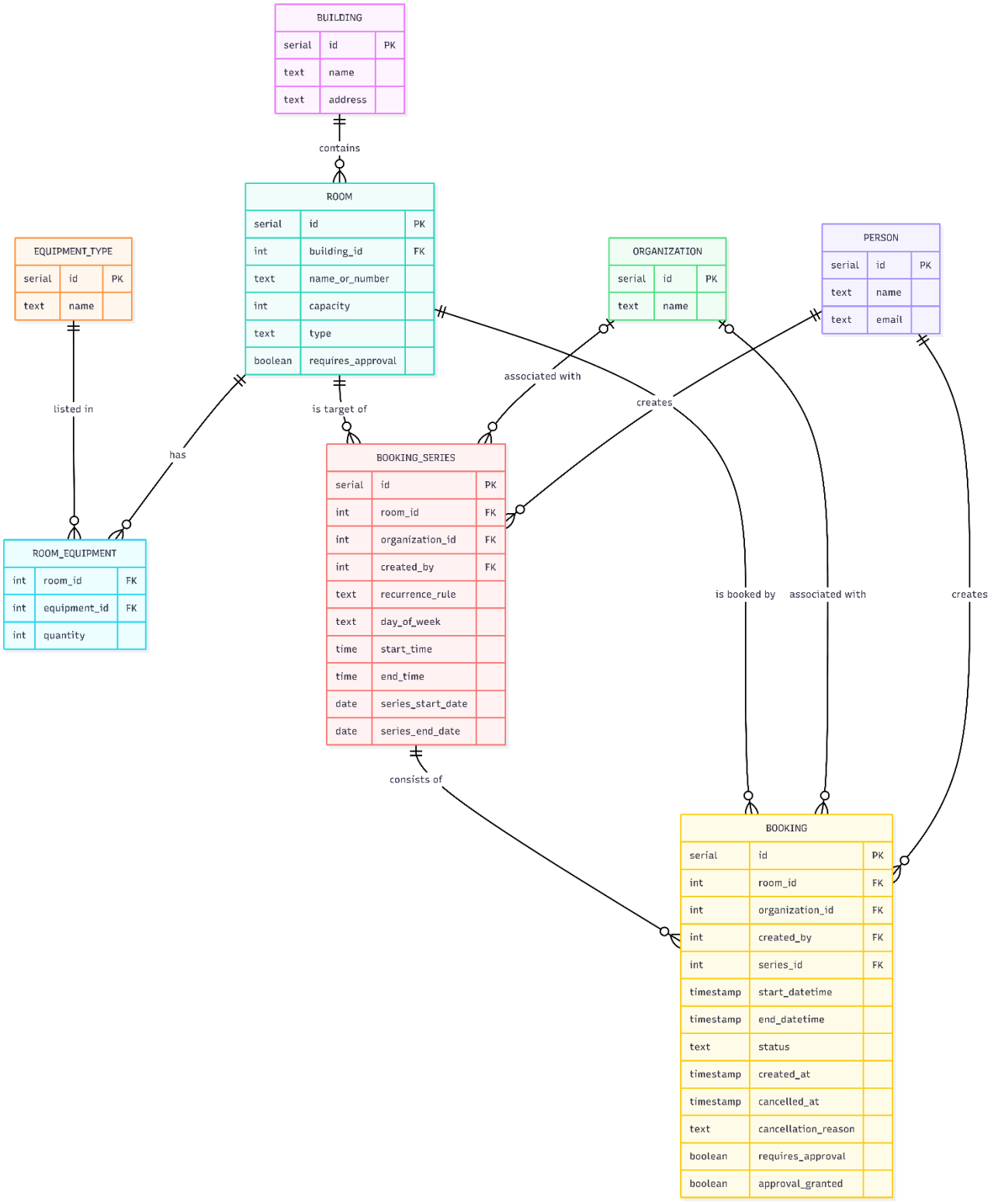
Conceptual Model & Relational Schema

Conceptual Model

ER Diagram

The diagram below is drawn in crow's foot notation. Each relationship line is read as:

- `||` : exactly one
- `o{` : zero or many
- `||--o{` : one to many (mandatory on the left, optional-many on the right)



Entities

Entity	Description
`BUILDING`	A university building that contains rooms
`ROOM`	A bookable room within a building
`EQUIPMENT_TYPE`	A type of equipment that may be present in rooms (e.g. projector)
`ROOM_EQUIPMENT`	Linking table, records which equipment types a room has, and quantity
`ORGANIZATION`	A student organization that makes bookings
`PERSON`	The individual who creates a booking or series
`BOOKING_SERIES`	The planning intent for a recurring booking (e.g. weekly Wednesday evenings)
`BOOKING`	One time slot, either a standalone booking or one occurrence of a series

Relationships and Cardinalities

Relationship	Cardinality	Description
BUILDING -> ROOM	1 to many	A building contains zero or more rooms; a room belongs to exactly one building
ROOM -> ROOM_EQUIPMENT	1 to many	A room has zero or more equipment entries
EQUIPMENT_TYPE -> ROOM_EQUIPMENT	1 to many	An equipment type appears in zero or more rooms
PERSON -> BOOKING	1 to many	A person creates zero or more bookings
PERSON -> BOOKING_SERIES	1 to many	A person creates zero or more booking series
ORGANIZATION -> BOOKING	0/1 to many	A booking may be associated with zero or one organization (nullable); an organization has zero or more bookings
ORGANIZATION -> BOOKING_SERIES	0/1 to many	A series may be associated with zero or one organization (nullable); an organization has zero or more series
ROOM -> BOOKING	1 to many	A room is the target of zero or more bookings
ROOM -> BOOKING_SERIES	1 to many	A room is the target of zero or more series
BOOKING_SERIES -> BOOKING	1 to many	A series consists of zero or more individual booking occurrences; a standalone booking has no series

Main Attributes per Entity

BUILDING

- `id` - surrogate primary key

- `name` - building name
- `address` - physical address

ROOM

- `id` - surrogate primary key
- `building_id` - FK to BUILDING
- `name_or_number` - room identifier within the building
- `capacity` - maximum number of people (must be > 0)
- `type` - one of: meeting room, lecture room, studio, workshop room
- `requires_approval` - whether the room requires special access approval (current policy)

EQUIPMENT_TYPE

- `id` - surrogate primary key
- `name` - equipment name (unique); e.g. projector, whiteboard, microphone, video camera

ROOM_EQUIPMENT (linking table)

- `room_id` - FK to ROOM (part of composite PK)
- `equipment_id` - FK to EQUIPMENT_TYPE (part of composite PK)
- `quantity` - how many units of this equipment type the room has (must be > 0)

ORGANIZATION

- `id` - surrogate primary key
- `name` - organization name (unique)

PERSON

- `id` - surrogate primary key
- `name` - full name
- `email` - contact email (unique); only personal data stored

BOOKING_SERIES

- `id` - surrogate primary key
- `room_id` - FK to ROOM
- `organization_id` - FK to ORGANIZATION; nullable (NULL if created by an individual)
- `created_by` - FK to PERSON
- `recurrence_rule` - descriptive rule string (e.g. "weekly", "biweekly")
- `day_of_week` - one of: Monday through Sunday
- `start_time` - time of day the slot starts
- `end_time` - time of day the slot ends (must be after start_time)
- `series_start_date` - date of first occurrence
- `series_end_date` - date of last occurrence (must be ≥ series_start_date)

BOOKING

- `id` - surrogate primary key
- `room_id` - FK to ROOM
- `organization_id` - FK to ORGANIZATION; nullable (NULL if booking made by an individual)
- `created_by` - FK to PERSON
- `series_id` - FK to BOOKING_SERIES; nullable - NULL for standalone bookings
- `start_datetime` - start of the booking slot
- `end_datetime` - end of the booking slot (must be after start_datetime; same calendar day)
- `status` - one of: pending, confirmed, cancelled, rejected
- `created_at` - timestamp when the booking was created
- `cancelled_at` - nullable; timestamp of cancellation
- `cancellation_reason` - nullable; short free text
- `requires_approval` - snapshot of the room's approval policy at booking creation time

- `approval_granted` - nullable Boolean: NULL if not required or pending; TRUE if approved; FALSE if rejected

Section 3 - Relational Schema

Tables Overview

Table	Primary Key	Foreign Keys
`building`	`id`	-
`room`	`id`	`building_id -> building`
`equipment_type`	`id`	-
`room_equipment`	`(room_id, equipment_id)`	`room_id -> room, equipment_id -> equipment_type`
`organization`	`id`	-
`person`	`id`	-
`booking_series`	`id`	`room_id -> room, organization_id -> organization, created_by -> person`
`booking`	`id`	`room_id -> room, organization_id -> organization, created_by -> person, series_id -> booking_series`

Table Definitions

building

Column	Type	Nullable	Notes
`id`	SERIAL	NOT NULL	Primary key
`name`	TEXT	NOT NULL	Building name
`address`	TEXT	NOT NULL	Physical address

room

Column	Type	Nullable	Notes
`id`	SERIAL	NOT NULL	Primary key
`building_id`	INT	NOT NULL	FK -> building(id)
`name_or_number`	TEXT	NOT NULL	Room identifier
`capacity`	INT	NOT NULL	Must be > 0
`type`	TEXT	NOT NULL	meeting room / lecture room / studio / workshop room
`requires_approval`	BOOLEAN	NOT NULL	Default FALSE; current room policy

Unique constraint: `(building_id, name_or_number)` - no two rooms in the same building share an identifier.

equipment_type

Column	Type	Nullable	Notes
id	SERIAL	NOT NULL	Primary key
name	TEXT	NOT NULL	Unique equipment type name

room_equipment

Column	Type	Nullable	Notes
room_id	INT	NOT NULL	FK -> room(id); part of composite PK
equipment_id	INT	NOT NULL	FK -> equipment_type(id); part of composite PK
quantity	INT	NOT NULL	Default 1; must be > 0

organization

Column	Type	Nullable	Notes
id	SERIAL	NOT NULL	Primary key
name	TEXT	NOT NULL	Unique organization name

person

Column	Type	Nullable	Notes
id	SERIAL	NOT NULL	Primary key
name	TEXT	NOT NULL	Full name
email	TEXT	NOT NULL	Unique; only personal field stored

booking_series

Column	Type	Nullable	Notes
id	SERIAL	NOT NULL	Primary key
room_id	INT	NOT NULL	FK -> room(id)
organization_id	INT	nullable	FK -> organization(id); NULL if series created by an individual
created_by	INT	NOT NULL	FK -> person(id)
recurrence_rule	TEXT	NOT NULL	e.g. "weekly", "biweekly"
day_of_week	TEXT	NOT NULL	Monday–Sunday
start_time	TIME	NOT NULL	Slot start time
end_time	TIME	NOT NULL	Slot end time; must be after start_time
series_start_date	DATE	NOT NULL	First occurrence date
series_end_date	DATE	NOT NULL	Last occurrence date; must be ≥ series_start_date

booking

Column	Type	Nullable	Notes
id	SERIAL	NOT NULL	Primary key
room_id	INT	NOT NULL	FK -> room(id)
organization_id	INT	nullable	FK -> organization(id); NULL if booking made by an individual
created_by	INT	NOT NULL	FK -> person(id);
series_id	INT	nullable	FK -> booking_series(id); NULL if standalone
start_datetime	TIMESTAMP	NOT NULL	Slot start
end_datetime	TIMESTAMP	NOT NULL	Slot end; must be after start; same calendar day
status	TEXT	NOT NULL	pending / confirmed / cancelled / rejected; default pending
created_at	TIMESTAMP	NOT NULL	Default NOW()
cancelled_at	TIMESTAMP	nullable	NULL if not cancelled
cancellation_reason	TEXT	nullable	NULL if not cancelled
requires_approval	BOOLEAN	NOT NULL	Snapshot from room at creation time; default FALSE
approval_granted	BOOLEAN	nullable	NULL = not required or pending; TRUE = approved; FALSE = rejected

Check Constraint on Approval State

requires_approval	approval_granted	Valid statuses
FALSE	NULL	any
TRUE	NULL	pending, cancelled
TRUE	TRUE	confirmed, cancelled
TRUE	FALSE	rejected

Additional Check Constraints

- `end_datetime > start_datetime`
- `DATE(start_datetime) = DATE(end_datetime)`: a booking cannot span midnight

Linking Tables

room_equipment

Resolves the many-to-many relationship between `room` and `equipment_type`. A room can have multiple equipment types; the same equipment type can appear in multiple rooms. The composite primary key `(room_id, equipment_id)` ensures no duplicate entries. The `quantity` column gives this linking table its own structure beyond the relationship itself.

Nullable Columns Summary

Table	Nullable Columns	Reason
booking	series_id	NULL for standalone bookings
booking	organization_id	NULL if booking made by an individual with no organization
booking	cancelled_at	NULL if not cancelled
booking	cancellation_reason	NULL if not cancelled; also optional even when cancelled
booking	approval_granted	NULL if approval not required or decision not yet made
booking_series	organization_id	NULL if series created by an individual with no organization

Design and Quality Review

1. Anomaly and Redundancy Analysis

The first redundancy in the schema is the duplication of the `requires_approval` flag. It is present in the `room` table (as the current room policy) and in the `booking` table (as a snapshot at the creation time of the booking).

This is a **reasonable denormalization**, but it still has the risk of an **update anomaly**: if the approval policy for a room changes, the `room.requires_approval` value is updated, but all historical `booking.requires_approval` snapshots remain unchanged. This is actually the intended behavior: historical bookings should not be changed retroactively. Future developers should not assume both columns are the same and try to synchronize them, corrupting the audit trail.

A secondary risk is in the `booking_series` table. It stores `room_id`, `organization_id`, and `created_by`. Each individual `booking` occurrence stores these same three foreign keys independently. If a booking later becomes associated with a different organization (for example, by correction), the series record and its occurrence records may become out of sync. There is no schema level constraint enforcing that values of `booking.organization_id` be the same as `booking_series.organization_id` for bookings belonging to that series, which is an **insertion/update anomaly risk** in the current design.

2. Functional Dependencies

FD 1: `id` → `requires_approval` (on the `booking` table)

> A booking's `requires_approval` value is determined entirely by its `id`: it is a snapshot of the room's approval policy taken at the moment the booking was created.

This FD is significant because it sits in tension with the analogous FD `id` → `requires_approval` on the `room` table. Both columns carry the same semantic label but serve different purposes. The booking-level snapshot breaks the normal expectation that approval policy is a property of the room alone. This is the root of the update anomaly described above: if `room.requires_approval` is updated, there is no cascade to historical bookings, which is correct by design but must be enforced by application logic rather than the schema.

FD 2: `(room_id, equipment_id)` → `quantity` (on the `room_equipment` table)

> The number of a particular type of equipment in a particular room is determined by the room/equipment type pair.

This FD shows that `room_equipment` is in **Boyce-Codd Normal Form (BCNF)**: the only determinant is the composite primary key `(room_id, equipment_id)`, and `quantity` depends on the whole key, not a part. There is no anomaly here. The FD is noted because it shows why we kept `quantity` in the linking table rather than on `equipment_type`. If it were to move there, it would incorrectly imply that quantity is a property of the equipment type globally, rather than of a specific room's inventory.

FD 3: `id → (room_id, organization_id, created_by, day_of_week, start_time, end_time)` (on `booking_series`)

> All descriptive attributes of a recurring booking series are determined by the series identifier alone.

This FD is clean and well-structured. However, it implies an implicit expected FD between `booking_series` and `booking`: for any booking where `id IS NOT NULL`, one would expect `booking.room_id = booking_series.room_id` and `booking.organization_id = booking_series.organization_id`. No CHECK constraint in the current schema enforces this expectation, which is the anomaly risk pointed out in Section 1. Theoretically, a booking occurrence could be inserted with a mismatched `room_id` relative to its parent series, and the schema would silently accept it.

3. Schema Revisions

The schema was reviewed against the three functional dependencies above and the anomaly risks identified. Two decisions were made:

No structural change to `requires_approval` duplication. The snapshot pattern is retained as-is. The risk is real but the design intent is correct: `room.requires_approval` reflects current policy; `booking.requires_approval` is an immutable historical record. The CHECK constraint on `booking` already enforces consistency between `requires_approval`, `approval_granted`, and `status` at the row level, which is the appropriate place for this logic. Adding a trigger to copy `room.requires_approval` into `booking.requires_approval` at INSERT time would be a reasonable application-layer safeguard, but is outside the scope of the schema definition itself.

A note is added regarding series-occurrence consistency. The schema does not currently enforce that `booking.room_id` matches `booking_series.room_id` when a `series_id` is present. In a production system, this would be addressed with either a trigger or an application-layer validation. For the scope of this project, the risk is documented as a known limitation rather than resolved structurally, since adding a cross-table CHECK

constraint enforcing this relationship is not supported natively in PostgreSQL without a trigger.

The `booking_series.organization_id` column is correctly nullable (to match `booking.organization_id`, which is also nullable: a person may book without an organization). This was verified to be consistent across both tables in the current schema.

4. Privacy Review

The `person` table stores only a name and a unique email address, the minimum required to identify who created a booking and to enable contact if a booking needs to be managed or cancelled. No phone numbers, student IDs, roles, or affiliations are stored, keeping personal data to a minimum. The `cancellation_reason` field on `booking` is the most significant privacy risk in the schema: it is a free-text field with no length constraint or content validation, and a user could enter personal or sensitive information (for example, a medical reason) that the system has no mechanism to detect or restrict. This field should be treated as potentially sensitive personal data in any access control policy, and operators should be advised to keep entries brief and non-identifying. Finally, because `person.id` is referenced by both `booking.created_by` and `booking_series.created_by`, deleting a person record is blocked by foreign key constraints. The system should define a clear data retention and deletion policy to handle right-to-erasure requests without orphaning booking history.

5. Rationale

Overview of the Design

The central idea of this design is to separate intent from occurrence. A student organization that books a room every Wednesday evening for a term has one planning intent but many individual time slots. The schema reflects this directly: `booking_series` stores the intent (the pattern, the room, the organization, the date range), while each `booking` row stores one concrete occurrence. A singular booking is simply a booking with no parent series (`series_id = NULL`). This separation means the system can answer both "does the organization hold Wednesday evenings this term?" and "was the 14 March session cancelled?" without conflating the two questions. Every other table in the schema supports this core structure: `building` and `room` describe the physical space being reserved; `organization` and `person` describe who is doing the reserving; `equipment_type` and `room_equipment` describe what is available in each space; and the `booking` table, with its status, approval, and cancellation fields, records what actually happened.

The Most Difficult Modeling Decision

The hardest decision was how to model the approval state. The case says only that some rooms require special access approval, and that the system should record whether approval

was required and whether it was granted. It says nothing about who approves, how many approval steps exist, or whether approvals can be partially granted.

We chose the simplest representation: a single nullable boolean `approval_granted` paired with a `requires_approval` flag. This captures all four meaningful states (no approval needed, pending, approved, rejected) without introducing an approver identity, a separate approval table, or a multi-step workflow that the case never asked for. The logic is encoded in a CHECK constraint that ties these two columns to the booking's `status`, making illegal combinations impossible at the database level: a booking cannot be `confirmed` while approval is still pending, and cannot be `rejected` unless approval was explicitly denied.

The tradeoff is that the schema cannot answer "who approved this booking?" or "how long did approval take?" If those questions become relevant later, the design would need a separate `approval` table with its own audit trail. For now, the scope was kept tight to what the case actually requires.

Choice of Primary Identifiers

Every table uses a surrogate primary key. The alternative was to use natural keys: a room's name or number within its building, an organization's name, a person's email address. Natural keys were rejected for three reasons.

First, they are mutable. An organization can be renamed, a person can change their email address, and a room can be renumbered when a building is renovated. A surrogate key never changes, so foreign key references across the schema remain stable regardless of what happens to the real-world data. Second, natural keys are often composite. A room is only uniquely identified by the combination of its building and its name or number, and using this as a primary key would propagate a two-column key into every table that references rooms, making joins more verbose and indexes larger. Third, surrogate keys keep the schema honest about what is actually unique: the `UNIQUE` constraint on `(building_id, name_or_number)` in the `room` table still enforces that no two rooms in the same building share an identifier, but this is a business rule separate from the row's identity. The same pattern applies to `organization.name` and `person.email`, both of which carry `UNIQUE` constraints independently of their surrogate keys.

Where Normalization Shaped the Design

The clearest example of normalization driving a structural decision is the extraction of `equipment_type` into its own table. Another approach would have been to store equipment as a comma-separated string on the `room` row, or to add boolean columns like `has_projector`, `has_whiteboard`. Both approaches violate First Normal Form: the first by storing multiple values in a single column, the second by encoding a variable-length list as a fixed set of columns that would require a schema change every time a new equipment type is introduced.

Extracting `equipment_type` into a standalone table and resolving the many-to-many relationship through `room_equipment` puts the design in Third Normal Form. The `quantity` column on `room_equipment` depends on the full composite key `(room_id, equipment_id)`, not on either foreign key alone. This means it belongs in the linking table, not on `equipment_type` (where it would imply a global quantity independent of the room) and not on `room` (where it could not vary by equipment type).

An Assumption Made Because the Case Was Incomplete

The case states that a recurring booking should be treatable "both as a single planning intent and as a series of individual occurrences", and that one occurrence may be cancelled or moved without affecting the rest of the series. It does not say what "moved" means in practice, whether a moved occurrence simply gets a new time, or whether it generates a replacement booking row, or whether the series itself is modified.

The assumption made here is that moving an occurrence is handled by cancelling the original booking row and creating a new standalone booking for the rescheduled time, with no changes to the `booking_series` record. This keeps the series record as a pure statement of original intent and treats each occurrence as an independent row that can be cancelled, confirmed, or rejected on its own merits. The consequence is that the series record will always show the original pattern, not reflecting specific reschedules. This was judged acceptable because the case frames the series as a planning intent, not a live schedule. If the university later requires the series record to stay in sync with actual occurrence times, a more complex event-sourcing approach would be needed, but that goes well beyond what the case describes.